

Junie: Your AI Coding Agent in JetBrains IDEs

Autonomous Coding with Safety Controls

Transform Your Development Workflow
with AI-Powered Automation

Four-Hour Hands-On Workshop

What You'll Master Today

Core Concepts

-  **Ask Mode:** Analysis & insights
-  **Code Mode:** Automated changes
-  **Guidelines:** Team conventions
-  **Safety:** Approvals & controls

Hands-On Labs

-  **Java:** Spring Boot APIs
-  **Python:** Refactoring & testing
-  **React:** Forms & validation
-  **MCP:** External tool integration

Prerequisites Check

- ✓ **JetBrains IDE** (IntelliJ IDEA, PyCharm, WebStorm, GoLand, PhpStorm, RustRover, RubyMine)
- ✓ **JetBrains AI** subscription (AI Credits model — 1 Credit = \$1 USD)
- ✓ **Development Environment** (Java 17+ / Python 3.8+ / Node 18+)



AI Free: 3 AI Credits/month — or use BYOK (Bring Your Own Key) with any provider



Quick Setup: We'll install Junie together in the first 5 minutes

Getting Started — Two Paths

IDE Plugin

1. AI Chat panel → Agent dropdown → Select ****Junie****
2. Or: Settings → Plugins → Search "Junie"
3. Start prompting

Junie CLI (Beta)

1. ``brew tap jetbrains/junie && brew install junie``
2. Or: ``curl -fsSL https://junie.jetbrains.com/install.sh | bash``
3. Run ``junie`` in any project

Available Models (LLM-agnostic):

GPT / GPT Codex

Opus / Sonnet

Gemini Pro / Flash

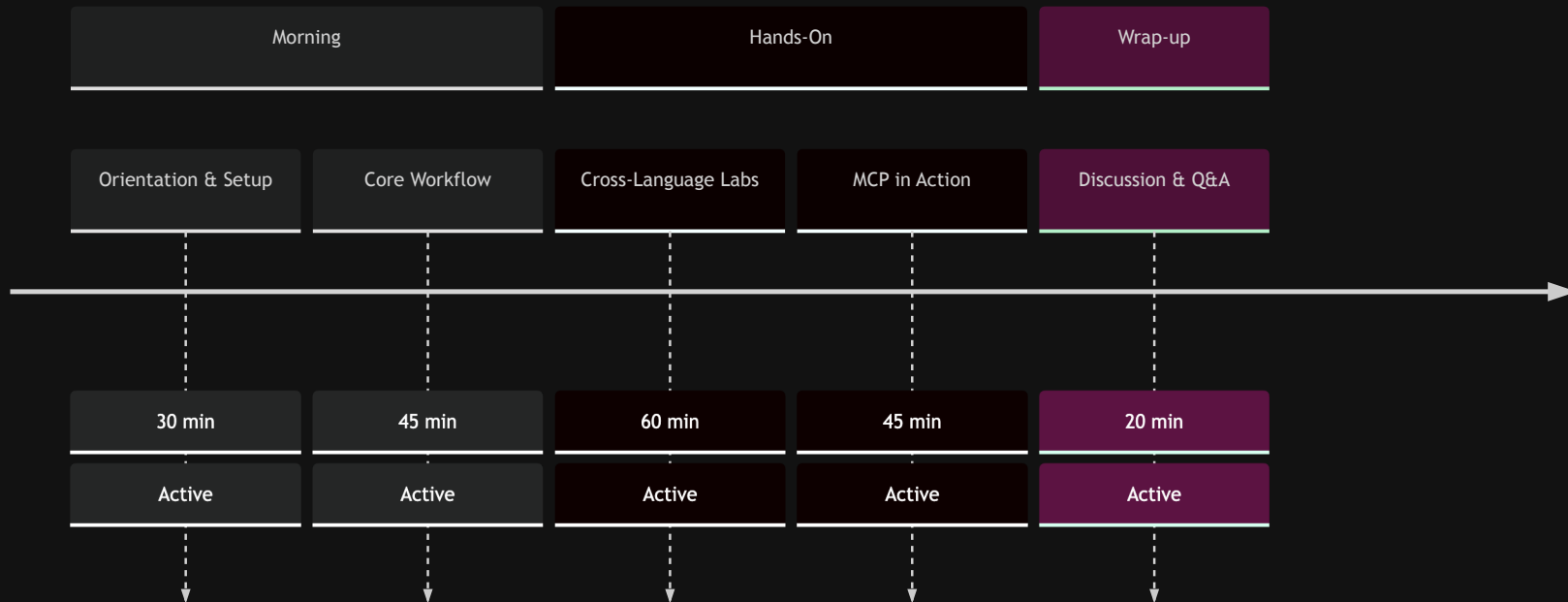
Grok

Aliases auto-update to latest versions



Today's Journey

Four-Hour Workshop Timeline

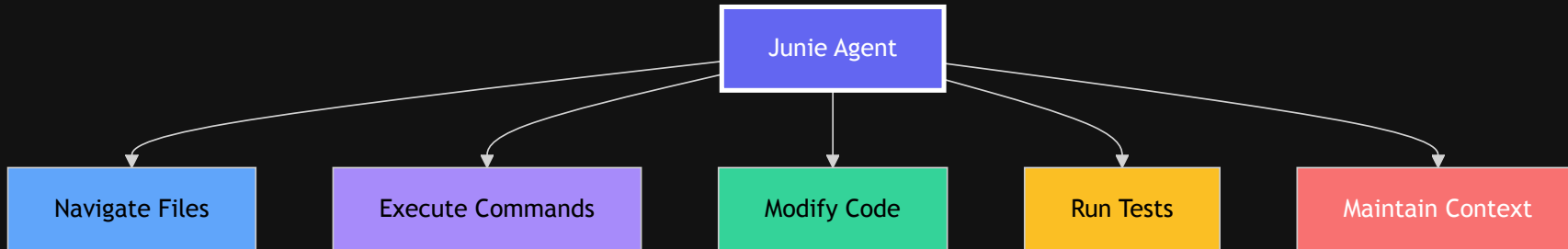


Break after Core Workflow segment



What is Junie?

JetBrains' AI-Powered Coding Agent



NEW

What's New in Junie (March 2026)

Junie CLI (Beta)

- Standalone terminal agent
- LLM-agnostic — BYOK support
- Plan mode, brave mode, prompt history
- One-click migration from Claude Code

AGENTS.md

- New open-standard guidelines format
- Replaces ``.junie/guidelines.md`` (legacy)
- Shared across AI coding agents
- Auto-import from other agents

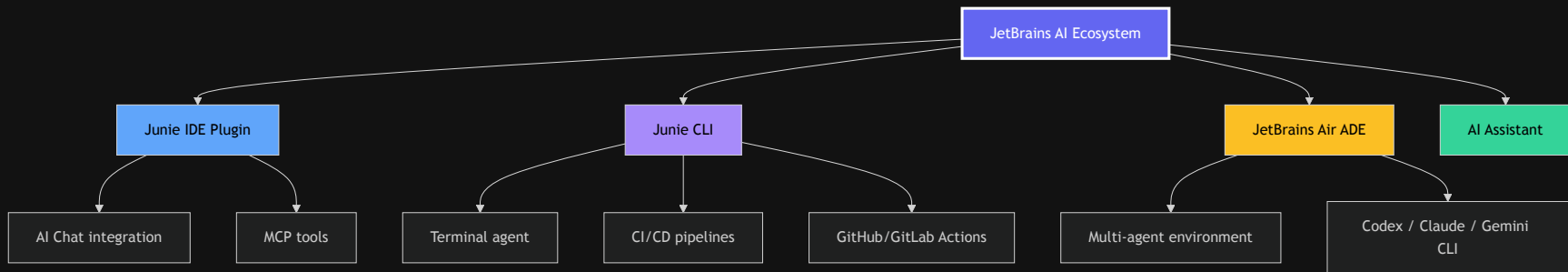
Agent Skills & Slash Commands

- Reusable task-oriented extensions
- ``.junie/skills/{name}/SKILL.md``
- Custom slash commands in CLI
- Project-level and user-level scopes

CI/CD & Beyond

- GitHub Action (``@junie-agent``)
- GitLab CI (``#junie`` in comments)
- Junie merged into AI Chat (Dec 2025)
- AI Credits pricing model

JetBrains AI Landscape (2026)



Air = agentic dev environment (macOS public preview, March 2026) built on Fleet codebase, using Agent Client Protocol (ACP)



Two Operating Modes



Ask Mode

Read-Only Analysis

- Explain complex code
- Analyze architecture
- Find bugs & issues
- Review security
- Check test coverage

Perfect for understanding before changing



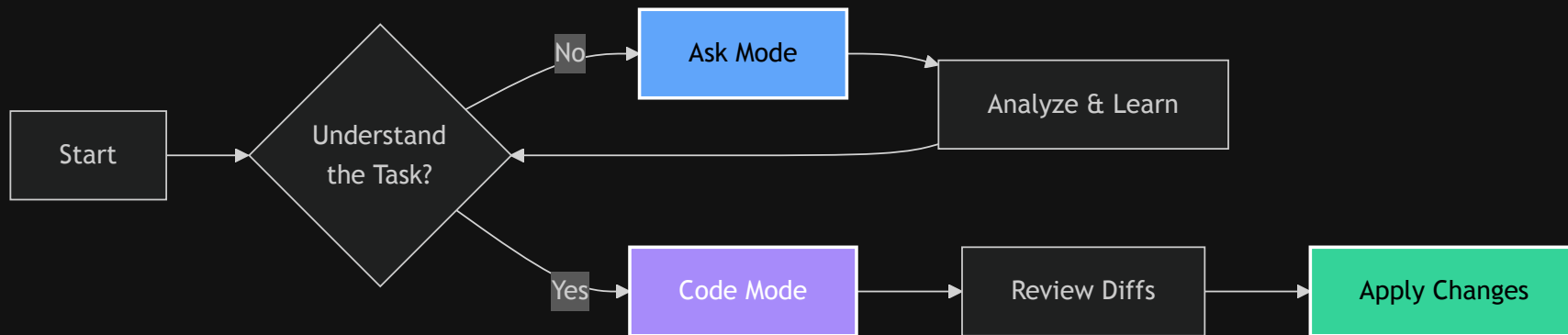
Code Mode

Make Changes

- Implement features
- Fix bugs
- Refactor code
- Generate tests
- Update dependencies

Executes multi-step plans with diffs

💡 Pro Workflow



✨ Always start with Ask mode for complex tasks



Project Guidelines

New: `.junie/AGENTS.md` (open standard) | **Legacy:** `.junie/guidelines.md` (still supported)

```
## Technology Stack
- Framework: Spring Boot 3.2
- Testing: JUnit 5 + AssertJ

## Conventions
- REST: /api/v1/{resource}
- DTOs: Java records
- Tests: Given-When-Then
```

Lookup order: `.junie/AGENTS.md` → `AGENTS.md` (project root) → `.junie/guidelines.md` (legacy)

✗ Without Guidelines

Inconsistent patterns

Multiple review cycles

Style conflicts

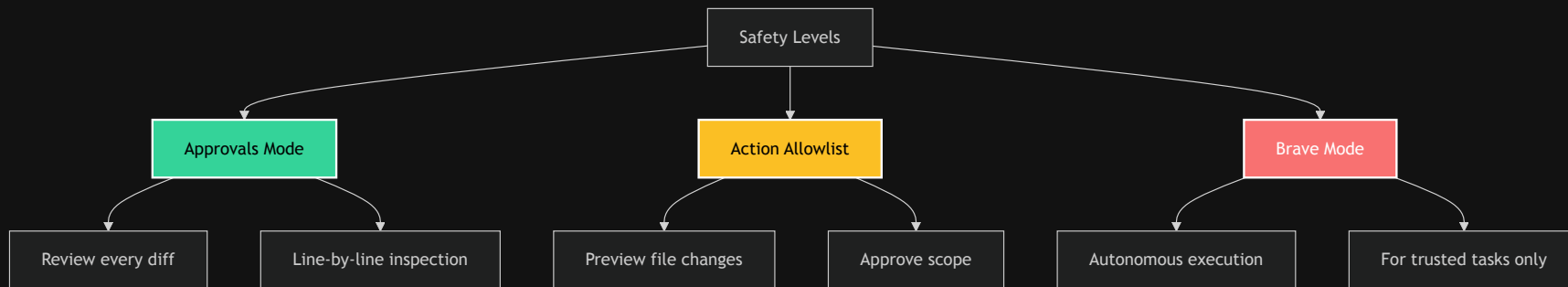
✓ With Guidelines

Consistent output

70% fewer reviews

Team alignment

Safety Controls



⚡ When to Use Each Mode

Mode	Good For	Not For
Approvals	• Critical business logic • First-time tasks • Learning Junie	• Repetitive formatting • Simple refactors
Allowlist	• Known file sets • Defined scope • Team reviews	• Exploratory changes • Unknown impact
Brave	• Test generation • Formatting • Documentation	• Production code • Database changes • First attempts

MCP: Model Context Protocol

Available Tools

context7

- Real-time library docs
- 1000+ libraries
- Version compatibility
- Optional API key (higher rate limits)

Playwright

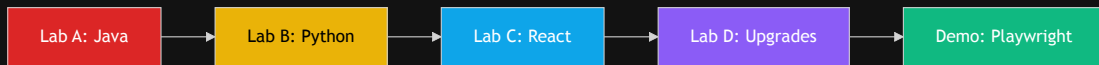
- E2E test generation
- Browser automation
- Accessibility-first
- TypeScript output

Configuration

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp"]
    },
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp-server"]
    }
  }
}
```

Settings → Junie → MCP → Edit mcp.json

Lab Overview



Timing:

- Lab A: 45-60 min
- Lab B: 30-45 min
- Lab C: 30-45 min
- Lab D: 20-30 min
- Demo: 20-30 min



Java

Spring Boot
REST APIs
JUnit + AssertJ



Python

PEP 8
Type hints
pytest



React

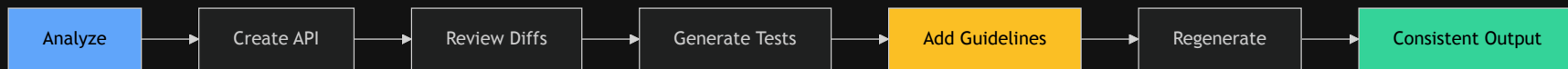
TypeScript
Forms
Testing Library



MCP

context7
Upgrades
Migration

Lab A: Spring Boot Journey



Without Guidelines

- Inconsistent patterns
- Various styles
- More review cycles

With Guidelines

- Consistent code
- Team standards
- Faster approval

Lab B: Python Transformation

Before 🤖

```
def calc(x,y,op):  
    if op=="add": return x+y  
    elif op=="sub": return x-y
```

- No type hints
- Poor naming
- No tests
- No docs

After ✨

```
def calculate(  
    x: float,  
    y: float,  
    operation: str  
) -> Optional[float]:  
    """Calculate result.  
  
    Args:  
        x: First number  
        y: Second number  
        operation: Operation  
  
    Returns:  
        Calculation result  
    """
```



Lab C: React Forms

```
interface RegistrationForm {  
  email: string;    // valid email  
  password: string; // min 8, special char  
  confirm: string;  // must match  
  terms: boolean;   // required  
}
```

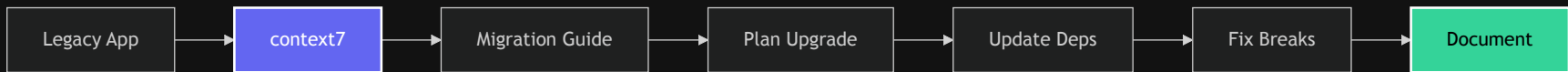
React Hook Form
Form state

Zod
Validation

Testing Library
90% coverage

Accessibility
ARIA compliant

Lab D: Smart Upgrades with context7



React 17 → React 18
Legacy Modern

Demo: Playwright Magic

Generate E2E Tests in 5 Minutes

```
test('successful login', async ({ page }) => {  
  const loginPage = new LoginPage(page);  
  await loginPage.goto();  
  await loginPage.login('user@example.com', 'pass');  
  
  await expect(page).toHaveURL('/dashboard');  
  await expect(page.getByRole('heading', { name: 'Welcome' })))  
    .toBeVisible();  
});
```

✓ Page Objects

✓ Accessibility

✓ TypeScript

✓ Retry Logic

💡 **MCP vs CLI:** IDE plugin needs MCP to reach the browser. CLI agents (Junie CLI, Claude Code, Codex) can run `npx playwright test` directly — full test runner, cheaper on tokens. Use MCP for interactive browser exploration; use CLI for running tests.

✂ Junie's Differentiators

Feature	Junie	Claude Code	Copilot Agent	Cursor
IDE + CLI + CI/CD	✅ All three	CLI only	IDE + CI	IDE only
LLM-agnostic	✅ BYOK	Claude native, others via routing	GPT native, others via routing	✅
Test execution	✅ Yes	✅ Yes	✅ Yes	✅ Yes
MCP tools	✅ Yes	✅ Yes	✅ Yes	✅ Yes
JetBrains native	✅ Full	via ACP	Plugin	✗

Common Patterns

TDD Flow

1. Ask: "What tests?"
2. Code: "Generate tests"
3. Code: "Implement"

Red → Green → Refactor

Safe Refactoring

1. Generate tests
2. Identify issues
3. Refactor safely

Tests stay green

Upgrades

1. context7 check
2. Update deps
3. Fix breaks

Systematic migration



CI/CD Integration — Junie GitHub Action

```
name: Junie Agent
on:
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]

jobs:
  junie:
    if: contains(github.event.comment.body, '@junie-agent')
    runs-on: ubuntu-latest
    steps:
      - uses: JetBrains/junie-github-action@v1
        with:
          junie-api-key: ${ secrets.JUNIE_API_KEY }
```

GitHub

`@junie-agent` in PRs/issues

GitLab

`#junie` in MR comments

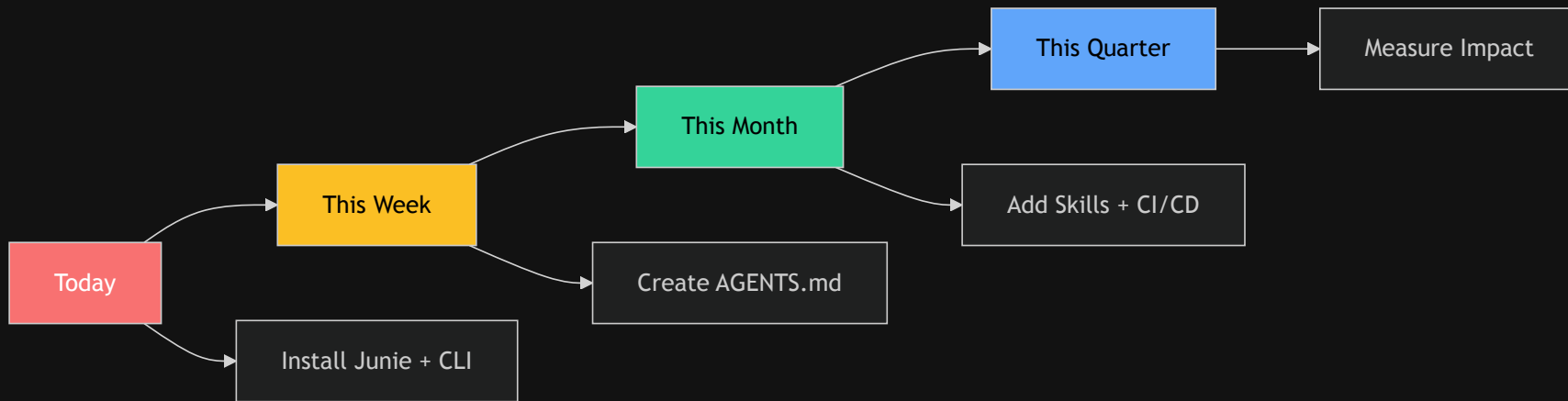
CLI Headless

`junie --auth="\$KEY" "prompt"`

✨ Key Takeaways

- ✓ **Ask before Code** - Understand first, implement second
- ✓ **Guidelines drive consistency** - Encode your team's DNA
- ✓ **Build trust gradually** - Approvals → Allowlist → Brave
- ✓ **MCP extends capabilities** - Leverage external tools
- ✓ **Safety first** - Review diffs, maintain control

Your Next Steps



 Start with Approvals Mode → Build Trust → Scale Up → Automate with CI/CD

Discussion Time

Let's Explore Together:

 IDE plugin vs CLI vs CI/CD — which fits your workflow?

 Where do you always want human review?

 How could AGENTS.md guidelines help your team?

 Which MCP tools and Agent Skills would you create?

 How will multi-agent environments (Air, ACP) change development?

Resources Hub

Documentation

-  [Junie Docs](#)
-  [Guidelines & Memory](#)
-  [Agent Skills](#)
-  [CLI Quickstart](#)

CI/CD

-  [GitHub Action](#)
-  [GitLab CI](#)

GitHub

-  [Guidelines Catalog](#)
-  [Context7 MCP](#)
-  [This repository](#)

Community

-  [JetBrains AI Slack](#)
-  [Stack Overflow: `jetbrains-junie`](#)
-  [Support team](#)

 Thank You!

AI amplifies good practices

 Remember:

**Invest in your guidelines today,
reap consistency forever!**

Questions? Let's explore together! 

About Ken Kousen

Ken Kousen

President, Kousen IT, Inc.

 ken.kousen@kousenit.com

 github.com/kousen

 [@talesfromthejarside](https://twitter.com/talesfromthejarside)

 kousenit.substack.com

 linkedin.com/in/kenkousen

 bsky.app/profile/kousenit.com

AI agents aren't here to replace developers—
they're here to make us **better** developers.

Bonus: Quick Reference

IDE Shortcuts

- `Ctrl+Alt+J` - Open Junie
 - `Ctrl+Enter` - Execute
 - `Esc` - Cancel operation ### CLI
- Shortcuts • `Shift+Tab` - Plan mode
- `Ctrl+B` - Brave mode
 - `Ctrl+R` - Prompt history
 - `Ctrl+T` - Session transcript

Common Commands

"Analyze this file"

"Add tests with 90% coverage"

"Refactor to best practices"

"Fix all type errors"

MCP Tools

```
{  
  "context7": "Library docs",  
  "playwright": "E2E tests",  
  "search": "Web search",  
  "custom": "Your tools"  
}
```

 Print this slide for your desk!